Trustworthy Pneumonia Detection XAI: The Comprehensive Technical Handbook

1. Introduction

This document serves as the absolute technical reference for the **Trustworthy Pneumonia Detection XAI** project. It goes beyond the conceptual framework, providing an exhaustive, module-by-module, function-by-function breakdown of the entire codebase. By reading this handbook, any engineer or researcher will understand exactly how the system achieves its robust safety guarantees.

---

2. Configuration & Core Settings (`configs/`)

The `configs` module centralizes paths and hyperparameters. - **`configs/config.py`**: Central configuration file. - `ensure_dirs()`: Utility function that guarantees output directories like `OUTPUT_DIR` and `MODEL_SAVE_DIR` exist before the pipeline runs, preventing runtime `FileNotFound` errors.

---

3. Data Ingestion & Transformation (`datasets/`)

Handles the Kaggle Chest X-Ray dataset, ensuring correct Train/Validation/Test splits with zero data leakage.

- **`datasets/dataset.py`**:

`ChestXRayDataset(Class)`: The primary PyTorch dataset subclass.

- `__init__()`: Initializes transformations and paths.
- `_load_samples()`: Recursively walks through the dataset folders (`NORMAL`, `PNEUMONIA`) to collect file paths and assign integer labels.
- `__getitem__()`: Reads the image using PIL, applies the Torchvision transforms, and returns the tensor and label.

`datasets/preprocessing.py`:

- `get_dataloaders()`: Factory function that wraps `ChestXRayDataset` in PyTorch `DataLoader` objects. It handles batching, shuffling (for training), and multiprocessing workers.

`_maybe_subset()`: Helps during development or demo modes by truncating the dataset to a small subset for rapid testing (e.g., `--quick` flag).

`datasets/transforms.py`:

`get_transforms()`: Returns standard ImageNet normalization (`mean=[0.485, 0.456, 0.406]`, `std=[0.229, 0.224, 0.225]`) and resizes inputs to 224x224. Data augmentation is defined here for the training split.

`datasets/split_dataset.py`:

- `split_kaggle_dataset()` & `split_folder()`: Utilities to re-partition raw data into strict train, val, and test splits so validation accuracy isn't artificially inflated by data leakage.

---

4. Model Architecture (`models/`)

Defines the convolutional neural networks and the ensemble logic.

- **`models/cnn_model.py`**:

- `ChestXRayCNN(Class)`: The wrapper class for all architectures.

- `__init__()`: Accepts backbone names like `resnet18`, `vgg16`. Instantiates the backbone, strips the standard classifier head, and replaces it with a `nn.Linear` layer that outputs a single logit. **Crucially**, it injects a `nn.Dropout(0.5)` layer right before the classification head to allow for Monte Carlo uncertainty sampling.

- `forward()`: Passes the tensor through the modified backbone.

- `get_last_conv_layer()`: Programmatically finds the final spatial layer needed for Grad-CAM.

`build_model()`: Factory method mapping strings (e.g., `resnet18`) to model constructions.

`models/ensemble.py`:

- `agreement_from_probabilities()`: Takes multiple model predictions and calculates an `AgreementScore`. If architectures strongly disagree on a case, the score drops, which subsequently degrades the overall `TrustScore`.

---

5. Explainability, Uncertainty & Trust (`explainability/`)

The core scientific contribution of this codebase. It forces the CNN to "show its work" and mathematically estimate its own ignorance.

- **`explainability/gradcam.py`**:
- `GradCAM(Class)`: Implements Gradient-weighted Class Activation Mapping.
  - `_register_hooks()` / `_remove_hooks()`: Attaches PyTorch hooks to the `target_layer` (last conv layer) to save activation maps during the forward pass and gradients during the backward pass.
  - `generate()`: Computes the channel-wise mean of the gradients, multiplies it with the activations, and passes it through a ReLU to highlight regions pushing the score towards "Pneumonia".

`overlay_heatmap()`: OpenCV utility that blends the 2D Grad-CAM array as a jet-colored heatmap over the original X-ray.

`explainability/attention_maps.py`:

- `LungSegmenter(Class)`: Contains logic to generate a lung mask.
  - `_geometric_prior_mask()`: If a U-Net is unavailable, it applies a central bounding box (heuristic prior) representing typical lung fields on a frontal chest X-ray.

`focus_overlap_score()`: Also known as the **FocusScore**. Computes the geometric intersection between the Grad-CAM high-activation areas and the lung mask.

`explainability/uncertainty.py`:

- `MCDropoutPredictor(Class)`:
  - `predict()`: Runs N (default 30) stochastic forward passes using Monte Carlo Dropout.
  - `uncertainty_level()`: Computes the variance across the passes. High variance implies the model is essentially guessing.

`ood_flag_from_confidence_uncertainty()`: Hard rule that flags a sample as `UNRELIABLE` (Out-of-Distribution) if the variance is too high while calibrated confidence is too low.

`explainability/trustscore.py`:

- `TrustEngine(Class)`: The orchestrator of safety.
  - `compute_trust_score()`: Calculates `TrustScore = Confidence * FocusScore * (1 - Uncertainty) * AgreementScore`.
  - `get_trust_level()`: Buckets the scalar score into `high`, `medium`, or `low` trust bands for clinician consumption.

---

6. Training & Calibration (`training/` & `evaluation/`)

- **`training/trainer.py` & `training/train.py`**:

`train_one_epoch()` & `evaluate_epoch()`: Standard PyTorch training loops using Binary Cross

Entropy with Logits Loss and the Adam optimizer. Tracks moving averages of loss and accuracy.

`evaluation/calibration.py`:

`TemperatureScaling(Class)`: Fixes the overconfidence of standard CNNs.

- `fit_temperature()`: Optimizes a single parameter T (Temperature) using L-BFGS on the validation set logits.
- `calibrate_probs()`: Applies Sigmoid(logit / T).

`evaluation/reliability.py` & **`evaluation/metrics.py`**:

- `compute_ece()`: Expected Calibration Error. Measures the difference between predicted confidence and empirical accuracy.
- `compute_classification_metrics()`: Calculates ROC-AUC, F1-Score, Precision, and Recall.

---

7. Inference Pipeline (`inference/`)

How the trained, calibrated, and trust-wrapped model is exposed to the outside world.

- **`inference/inference.py`**:

`run_pipeline()`: The master function for a single image. It executes:

1. Standard Preprocessing.
2. Monte Carlo Dropout passes for Uncertainty.
3. Deterministic pass for Calibrated Confidence.
4. Grad-CAM generation & FocusScore calculation.
5. TrustEngine evaluation. Returns a unified dictionary containing all metrics, flags, and overlays.

`inference/predict.py` & **`inference/api.py`**:

- CLI and programmatic API wrappers around `run_pipeline()`.

---

8. Summary

By meticulously dividing the codebase into these modular domains (Classification, Calibration, Explainability, and Uncertainty), the system mathematically guarantees that predictions are not just accurate, but also interpretable, conservatively confident, and inherently trustworthy.